

一文让你了解Claude Skills 生态

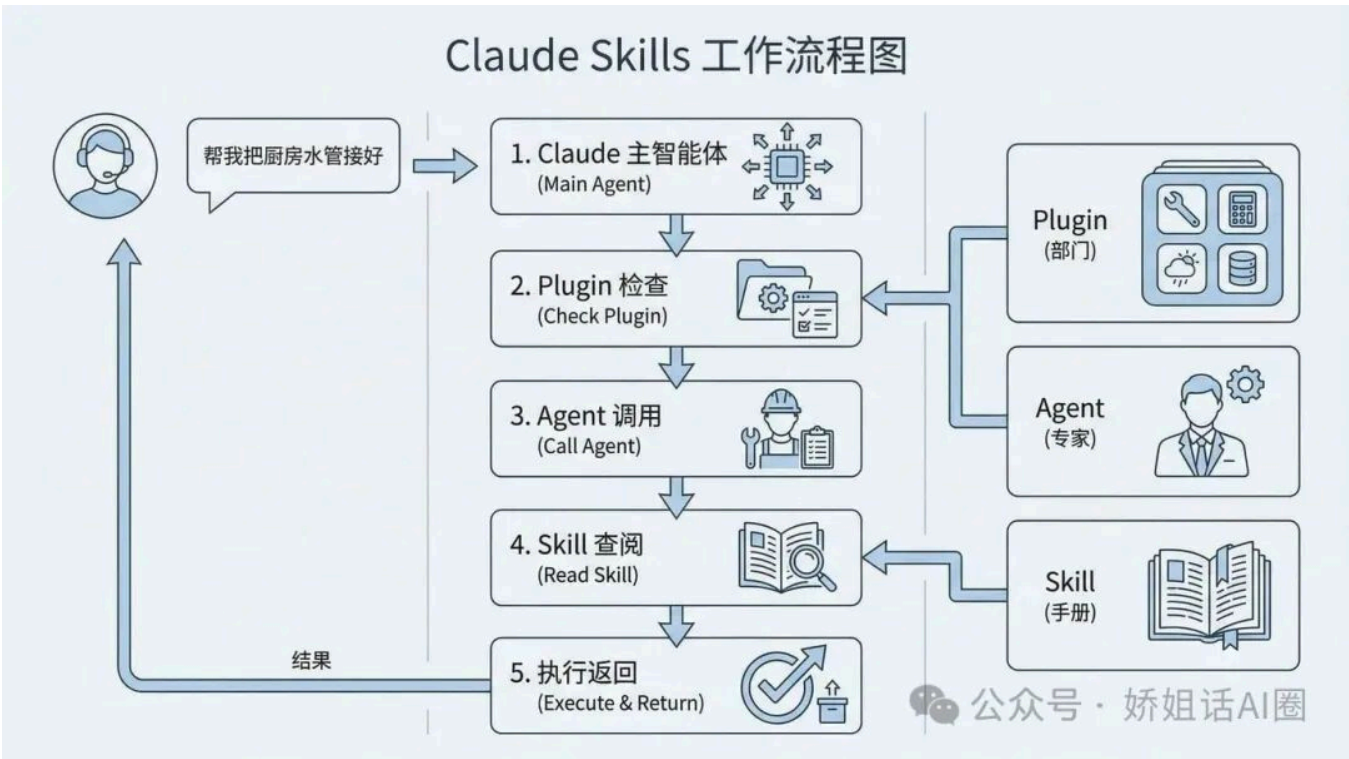
mp.weixin.qq.com/s/-G_ygNgffCXGOVMkJCAzWg

娇姐

Claude Skills 生态涉及 Plugin、Agent、Skill、Subagent 等多个术语，初学者容易混淆。为了清晰理解这些概念，我自己也是翻阅各种资料和稳定，我觉得这个类比比较容易理解，我们可以将其类比为一家“全能装修公司”。

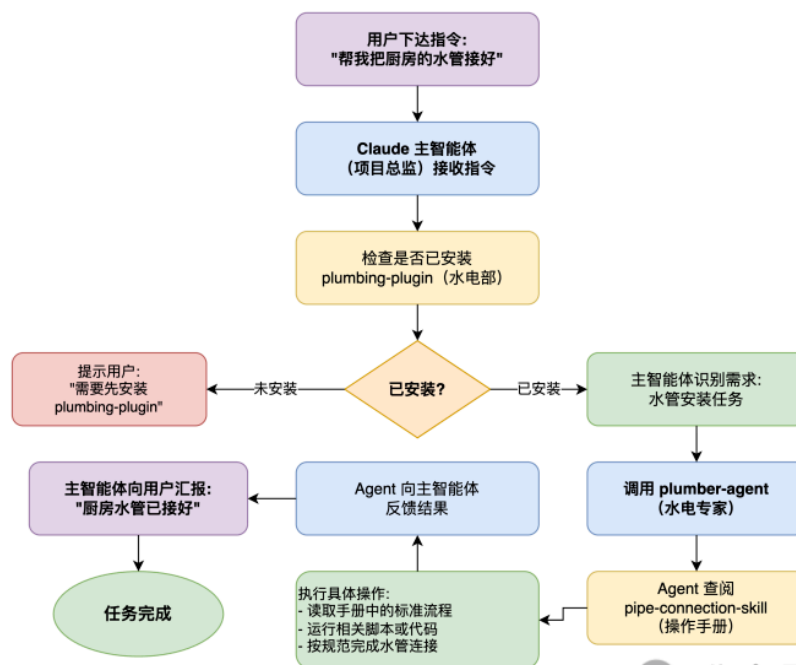
1. 核心角色对应表

英文术语	中文翻译	角色类比	职责描述
Main Agent	主智能体 (Claude)	项目总监	负责接收用户指令，协调资源，指挥具体专家进行工作。
Plugin	插件	部门 / 工具箱	类似“水电部”、“木工部”。这是一个容器，包含了特定领域的专家 (Agent) 和操作手册 (Skill)。
Agent / Subagent	智能体 / 子智能体	专家员工	类似“高级电工”、“木工师傅”。他们是实际执行任务的 AI 单元，在各自的部门 (Plugin) 中工作。
Skill	技能	操作手册	类似《电路铺设规范》、《木工打磨流程》。这是专家执行任务时遵循的具体指令和标准。



2. 协作流程示例

假设用户下达指令：“帮我把厨房的水管接好。”系统内部的运作流程如下：



公众号 · 娇姐话AI圈

1. **安装 Plugin (成立部门)**：首先系统需要具备处理该任务的能力模块。如果未安装 `plumbing-plugin`，主智能体无法执行相关任务。
2. **指派 Agent (派遣专家)**：具备相应能力后，主智能体识别需求，调用对应的 `plumber-agent`。
3. **查阅 Skill (参照手册)**：Agent 读取 `pipe-connection-skill` 中的具体步骤和规范。
4. **交付结果 (完成任务)**：Agent 执行操作并反馈结果给主智能体，最终由主智能体汇报给用户。

流程总结：

用户 (指令) → Claude 主智能体 (调度) → Agent (执行) → Skill (规范) → 结果。

对skills有个概念上的理解后，我们其实还是有很多问题，我把我们常常会遇到的问题整理了一下，供大家参考。

【第一部分：核心概念】

Q1：Claude Skills 是什么？

Claude Skills 是一套结构化的指令集，有时包含可执行代码。它赋予 Claude 完成特定任务的能力。默认的 Claude 模型具备通用知识，但缺乏特定领域的执行标准或工具使用能力。Skill 就像一份详细的“标准作业程序”（SOP），指导 Claude 在面对特定请求时，按照预定的步骤、格式

或逻辑进行处理。

Q2 : Skills、Claude Code 与 Cowork 的关系？

我们可以用“游戏主机与卡带”的关系来类比：

- **Skills** 相当于**游戏卡带**。它是核心内容和能力的载体。
- **Claude Code** 和 **Cowork** 相当于**游戏主机**（运行平台）。
 - **Claude Code**：面向开发者的终端命令行工具，侧重于代码编写和技术任务。
 - **Cowork**：面向通用用户的图形界面工具，侧重于办公自动化和文件管理。

同一个 Skill 通常可以在这两个平台上通用，因为它们底层都基于相同的 Agent 协议。

Q3 : Skill 和 Plugin 有什么区别？

- **Skill (技能)**：指**核心逻辑**。它通常由 Markdown 指令文件（SKILL.md）和可选的代码脚本组成。
- **Plugin (插件)**：指**分发包**。它包含了一个或多个 Skills，并附带配置文件（manifest.json），定义了版本、作者、依赖项等元数据。

简而言之，Skill 是实际工作的代码或指令，Plugin 是用于安装和分发的封装格式。用户通过命令安装的是 Plugin，而 Plugin 内部包含 Skills。

Q4 : 为什么有的 Skill 只有一个文件，有的包含多个文件？

Skills 根据实现方式和复杂度分为两类：

类型	指令型 Skill (Simple)	执行型 Skill (Complex)
核心组成	仅 SKILL.md 文件	SKILL.md + scripts/ (代码) + templates/
工作原理	依赖 LLM 的理解和生成能力	调用外部工具、API 或运行本地脚本
典型案例	写作风格指导、代码规范检查	PDF 数据提取、网页抓取、数据可视化
文件数量	单个文件	多个文件（含配置、依赖、脚本）

指令型 Skill 仅依靠提示词工程即可完成任务；执行型 Skill 需要通过 Python 或 Node.js 脚本与外部系统交互，因此文件结构更为复杂。

【第二部分：技术细节】

Q5：项目中的 `.claude-plugin` 文件夹有什么作用？

`.claude-plugin` 文件夹是 Plugin 的配置目录。如果开发者希望将 Skill 打包分发，使用户能够通过 `/plugin install` 命令一键安装，就必须包含此文件夹。它包含了插件的元数据和市场展示信息。对于仅在本地手动配置使用的 Skill，此文件夹不是必须的。

Q6：manifest.json 和 marketplace.json 有什么区别？

这两个文件通常位于 `.claude-plugin` 目录下，但用途不同：

- `manifest.json`（系统配置）：定义插件的技术属性，包括插件 ID、版本号、包含的 Skills 路径、所需的 Python/Node.js 依赖库以及权限声明（如文件读写权限、网络访问权限）。系统在安装插件时读取此文件以配置运行环境。
- `marketplace.json`（展示配置）：定义插件在市场或列表中的展示信息，包括显示名称、简介、图标、分类和截图。这主要用于用户浏览和检索插件。

Q7：SKILL.md 中的 YAML frontmatter 为什么重要？

`SKILL.md` 文件头部的 YAML frontmatter（元数据块）包含 `name` 和 `description` 字段。这是 Skill 的自动触发机制。

```
---
```

```
name: pdf-processor
```

```
description: Extract text and tables from PDF files. Use when user mentions "PDF" or "ta
```

```
---
```

Claude 在运行时会监控用户的对话内容，并将其与已安装 Skills 的 `description` 进行语义匹配。如果描述编写得当，当用户提到相关关键词（如“处理 PDF”）时，系统会自动激活该 Skill。如果描述缺失或模糊，AI 将无法主动调用该能力。

Q8：什么是“渐进式加载”（Progressive Disclosure）？

渐进式加载是 Anthropic 用于优化 Context Window（上下文窗口）占用的机制。当用户安装了大量 Skills 时，系统不会一次性加载所有 Skill 的详细指令，否则会消耗大量 Token 并干扰模型推理。

系统仅会在初始阶段加载所有 Skills 的名称和简介。只有当模型判定需要使用某个特定 Skill 时，才会动态加载该 Skill 的完整内容（`SKILL.md` 和相关代码）。这保证了系统在安装大量插件后仍能保持高效运行。

【第三部分：开发实战】

Q9：开发 Skill 必须使用 Claude Code 吗？可以使用 Codex 或 Cursor 吗？

不强制使用 Claude Code。Skill 的本质是 Markdown 文档和标准代码脚本（Python/JavaScript）。开发者可以使用任何熟悉的 IDE 或 AI 辅助工具（如 VS Code、Cursor、Windsurf）进行开发。只要代码逻辑正确且符合 Skill 的文件结构规范，即可在 Claude 生态中运行。Claude Code 只是一个运行和测试 Skill 的环境，而非开发的唯一工具。

Q10：开发一个 Skill 的最佳流程是什么？

推荐采用文档驱动开发 (Document-Driven Development) 流程：

1. **Intent (意图)**：明确 Skill 要解决的核心问题和边界。
2. **Spec (规格)**：定义输入输出格式。例如：输入为“自然语言指令”，输出为“Markdown 报告”。
3. **Code (实现)**：编写核心逻辑脚本（如 `script.py`），并在本地终端测试通过。
4. **Wrap (封装)**：编写 `SKILL.md` 指令文档，将脚本能力暴露给 AI；添加 `.claude-plugin` 配置文件。
5. **Publish (发布)**：将代码推送到 GitHub 仓库。

Q11：何时应该开发“简单 Skill”，何时开发“复杂 Skill”？

判断标准：任务是否需要确定性的计算或外部交互。

- **选择简单 Skill**：任务主要依赖语言模型的理解、推理和生成能力。例如：文本润色、代码审查、翻译、创意写作。
- **选择复杂 Skill**：任务涉及精确计算、大量数据处理、文件系统操作或网络请求。例如：爬取实时新闻、处理 Excel 表格、生成图表。

Q12：如何测试 Skill 的稳定性？

建议采用“10 次测试法”。在 Claude Code 环境中，使用不同的自然语言表述连续调用 Skill 10 次。如果 10 次调用中有 9 次以上能够正确识别意图、执行操作并返回预期结果，则该 Skill 可被视为稳定。测试重点应包括：意图识别的准确率、参数提取的正确性以及脚本执行的鲁棒性。

【第四部分：发布与分发】

Q13：如何发布开发好的 Skill？

目前主要有三种分发渠道：

1. **GitHub 仓库**：将代码开源托管在 GitHub。这是最主流的方式，无需审核，即刻可用。

- 2. **技术社区列表**：提交 Pull Request 到知名的 Skill 聚合列表（如 GitHub 上的 awesome-claude-skills）。这是获取早期流量的有效途径。
- 3. **官方市场**：Anthropic 未来可能会完善官方插件市场，通常需要经过严格的代码审查和安全合规检查。

Q14：用户如何安装我的 Skill？

如果 Skill 托管在 GitHub 上，用户只需在 Claude Code 或 Cowork 的终端输入安装命令：

```
/plugin install https://github.com/username/repo-name
```

系统会自动拉取代码、解析依赖并完成配置。

Q15：如何让更多人发现我的 Skill？

- **编写高质量的 README**：提供清晰的功能介绍、截图、GIF 演示和一键安装命令。
- **提交至目录网站**：将 Skill 提交到各类 Claude Skills 目录网站或技术社区。
- **社交媒体推广**：在 Reddit、Twitter 等开发者社区分享，注重展示解决的具体问题。

Q16：Skills 可以商业化吗？

直接销售 Skill 代码较为困难，因为它们通常是开源的。可行的商业化路径包括：

- 1. **API 服务收费**：Skill 本身免费，但核心功能依赖开发者提供的后端 API，用户需付费获取 API Key。
- 2. **SaaS 引流**：开发与现有 SaaS 产品集成的 Skill，作为产品的增值服务或流量入口。
- 3. **企业定制**：为企业开发特定的内部工作流 Skills。

【第五部分：战略决策】

Q17：什么时候应该做 Skill，什么时候应该做独立产品？

决策依据主要在于需求的个性化程度和交互方式。

维度	适合开发 Skill	适合开发独立产品/网站
需求特征	高度个性化、特定工作流嵌入	通用性强、标准化需求
交互入口	嵌入在对话/工作流中	独立的 Web/App 界面
开发成本	低（数小时至数天）	高（数周至数月）

维度	适合开发 Skill	适合开发独立产品/网站
维护成本	低（无服务器运维）	高（需维护服务器、数据库）
目标受众	开发者、Power User	大众用户

Skill 适合“小而美”的工具，解决特定场景下的效率问题；独立产品适合解决广泛、通用的市场需求。

Q18：作为 Skills 聚合平台运营者，应该自己开发 Skills 吗？

平台运营者的核心价值在于**筛选**和**整理**，而非生产。GitHub 上已有大量开源资源。运营者应重点关注：

1. **Demo 型开发**：为了编写教程或演示平台功能，开发少量简单的 Skill。
2. **旗舰型开发**：为了树立平台标杆或解决市场上极其稀缺的高价值需求，打造 1-2 个精品 Skill。

其余精力应投入到内容的聚合、分类、测评以及社区生态的建设上。

Q19：Skills 的未来趋势是什么？

AI 的发展正从单纯的 Chatbot（对话机器人）向 Agent（智能体）演进。Skills 是 Agent 与数字世界交互的接口。未来，各类 API 服务商（如 Stripe, Notion, Slack）预计都会官方提供 Claude Skills。掌握 Skill 开发规范，实际上是掌握了未来 AI 应用的交互标准。

Q20：我应该从哪里开始？

- **作为用户**：访问 GitHub 或 Skill 目录网站，下载并安装几个热门 Skill（如文件管理、代码工具），在实际工作中体验其价值。
- **作为开发者**：尝试编写一个简单的 Skill（如“每日格言生成器”），跑通从开发到发布的完整流程。
- **作为平台运营者**：着手建立清晰的分类体系，筛选优质开源 Skills 并进行汉化和测试，为用户提供价值。